

AI-Assisted Compiler Error Diagnosis System

Final Project Report

CS1202 — Semester Project

Student MADDELA RAJEEV

Roll No. 24CSB0B40

Course CS1202

Abstract

This report presents the complete development of an AI-Assisted Compiler Error Diagnosis System for C++ programs. The system addresses a fundamental problem faced by beginner programmers: raw GCC/G++ compiler error messages are technically correct but difficult to interpret. The proposed solution integrates a machine-learning classifier trained on 129 real compiler errors extracted from 82 hand-crafted C++ programs, a feature-extraction pipeline producing 28 structured ML features, and a rule-based fix-suggestion engine. The trained Random Forest model achieves 86% classification accuracy across 8 error categories, with a cross-validation mean of 87.7%. End-to-end testing on 16 unseen programs yields a 93.75% pass rate. The system classifies errors in under 1 second using less than 150 MB of memory, well within all SRS-defined targets. The project was executed across 14 structured weekly phases, with deliverables documented at each stage.

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Project Objectives	4
1.3	Scope	4
1.4	Report Structure	4
2	Literature Survey and Research Gap	5
3	System Requirements (SRS Summary)	5
3.1	Functional Requirements	5
3.2	Non-Functional Requirements and Targets	5
4	System Architecture	6
4.1	High-Level Design	6
4.2	Module Interaction	6
4.3	Technology Stack	6
5	Phases 1–2: Problem Definition, Literature Survey, and SRS	7
5.1	Weeks 1–2: Problem Definition and Literature Survey	7
5.2	Weeks 3–4: SRS and Architecture	7
6	Phase 3: Data Collection and Feature Extraction (Weeks 5–6)	7
6.1	Week 5: Error Collection	7
6.2	Week 6: Feature Extraction	8
7	Phase 4: Dataset Preparation and Model Training (Weeks 7–8)	8
7.1	Week 7: Dataset Preparation	8
7.2	Week 8: Model Training	8
8	Phase 5: Classification Integration and Fix Engine (Weeks 9–10)	9
8.1	Week 9: Classification Integration	9
8.2	Week 10: Fix Suggestion Engine	9
9	Phase 6: Security, Robustness, and Performance (Weeks 11–12)	10
9.1	Week 11: Security and Robustness Testing	10
9.2	Week 12: Performance Optimisation	10
10	Week 13: System Testing and Validation	10
10.1	End-to-End Test Summary	10
10.2	Single Failure Analysis	11
10.3	Overall Validation Against SRS Targets	11
11	Source Code Overview	11
12	Conclusion and Future Work	12
12.1	Conclusion	12
12.2	Key Contributions	12
12.3	Limitations	12

12.4 Future Work	12
A Source Module Descriptions	13
B SRS Non-Functional Requirements — Full Table	13

Introduction

Background and Motivation

Compiler error messages produced by tools such as GCC and G++ serve as the primary feedback mechanism for C++ programmers. While these messages are technically precise, they assume a level of familiarity with language syntax and compiler internals that beginner students often lack. Typical raw messages such as `expected ';' before 'double' or 'score' was not declared in this scope` convey the location and nature of the error but provide no guidance on why it occurred or how to fix it.

This gap between machine-generated diagnostics and human-comprehensible explanations creates a significant barrier in educational settings. Students spend disproportionate time deciphering error messages rather than learning the underlying programming concepts. An intelligent intermediary layer — one that classifies errors, explains their cause, and suggests targeted fixes — can meaningfully accelerate the learning process.

Project Objectives

The objectives of this project are:

- To collect a representative dataset of real C++ compiler errors from diverse, hand-written programs.
- To extract structured features from compiler output that capture the distinguishing characteristics of each error category.
- To train and evaluate machine learning models capable of classifying errors into 8 defined categories.
- To develop a fix-suggestion engine that maps classified errors to actionable, beginner-friendly repair steps.
- To integrate all components into a single end-to-end diagnosis pipeline.
- To validate the system against unseen programs and confirm it meets all SRS-defined requirements.

Scope

The system targets C++ programs compiled with GCC/G++. It supports 8 error categories: Missing Semicolon, Undeclared Variable, Type Mismatch, Unmatched Braces, Unmatched Parentheses, Missing Return Statement, Invalid Syntax in Declarations, and Missing Header Include. The system operates as a command-line tool that wraps compiler invocation and augments its output.

Report Structure

This report is organised as follows. Section 2 surveys related work. Section 3 describes the system requirements. Section 4 presents the system architecture. Sections 5 through 10 detail the development phases covering data collection, feature extraction, model training, integration, security, and performance. Section 11 presents Week 13 testing and validation results. Section 12 summarises conclusions and future work. Appendix A lists the source code modules.

Literature Survey and Research Gap

Compiler error diagnosis has been an active research area for several decades. Early systems such as IOTA (1976) produced simplified error messages for teaching PASCAL, establishing the precedent of translating raw compiler output into student-friendly language. Rule-based systems dominated until the 2010s, mapping fixed error patterns to canned explanations.

More recent work has applied machine learning to code error detection. DeepFix (2017) used a sequence-to-sequence neural network to predict single-line fixes for common C errors. Ahmed et al. (2018) demonstrated that Random Forest classifiers trained on token-level features outperform rule-based systems for error category prediction. CodeBERT and similar transformer-based models have shown strong performance on code understanding tasks but require significant computational resources.

The research gap addressed by this project is the absence of a lightweight, end-to-end C++ compiler error assistant that: (a) operates entirely locally without internet access, (b) provides category, confidence score, explanation, and step-by-step fix in a single output, and (c) is trained on real GCC/G++ output rather than synthesised datasets. Existing tools either focus on Python errors, require cloud APIs, or provide explanations only, without a classification confidence score.

System Requirements (SRS Summary)

Functional Requirements

- FR-01: Accept a C++ source file as input and compile it using GCC/G++.
- FR-02: Parse raw compiler error output and extract structured features.
- FR-03: Classify each error into one of 8 defined categories using a trained ML model.
- FR-04: Report the predicted category and a confidence score (0–100%).
- FR-05: Provide a plain-English explanation of the error cause.
- FR-06: Output step-by-step fix instructions with a corrected code example.
- FR-07: Support all 8 error categories defined in the SRS.

Non-Functional Requirements and Targets

Requirement	Target	Achieved
Classification Accuracy	$\geq 85\%$	86%
Cross-Validation Mean	$\geq 85\%$	87.7%
Response Time (per file)	< 3 seconds	< 1 second
Memory Usage	< 500 MB	< 150 MB
End-to-End Pass Rate	$\geq 85\%$	93.75% (15/16)
Robustness Tests	18/18 pass	18/18 (100%)
Security (dangerous patterns)	0 patterns	0 patterns

Requirement	Target	Achieved
Error Categories Supported	8 categories	8 categories

System Architecture

High-Level Design

The system follows a five-stage pipeline:

1. **Input:** A C++ source file (`.cpp`) is supplied to the system.
2. **Compilation:** GCC/G++ compiles the file and captures raw error output.
3. **Feature Extraction:** `extract_features.py` parses the raw error and computes 28 binary/numeric features.
4. **Classification:** The trained `best_model.pkl` (Random Forest) predicts the error category and confidence.
5. **Fix Suggestion:** `fix_suggestions.py` maps the predicted category to a structured explanation and repair steps.
6. **Output:** `main.py` assembles and prints the full enhanced diagnosis.

Module Interaction

Module	Responsibility
<code>main.py</code>	Entry point; orchestrates all modules; prints final output
<code>classify_errors.py</code>	Loads <code>best_model.pkl</code> ; calls feature extractor; returns (category, confidence)
<code>fix_suggestions.py</code>	Dictionary-based lookup: category → explanation + fix steps + code example
<code>extract_features.py</code>	Regex-based parser producing 28-feature vector from raw compiler text
<code>train_model.py</code>	Offline script: loads <code>features_dataset.csv</code> , trains 4 models, saves best
<code>collect_errors.py</code>	Compiles <code>.cpp</code> files in batch; writes <code>collected_errors.csv</code>
<code>best_model.pkl</code>	Serialised Random Forest classifier (scikit-learn)

Technology Stack

Python 3.x, scikit-learn (Random Forest, Decision Tree, SVM, Naive Bayes), pandas, NumPy, GCC/G++ (system compiler), subprocess module for compiler invocation, pickle for model serialisation.

Phases 1–2: Problem Definition, Literature Survey, and SRS

Weeks 1–2: Problem Definition and Literature Survey

Week 1 established the project scope through a detailed study of the GCC/G++ compilation workflow, error taxonomy, and existing educational debugging tools. The compiler pipeline (preprocessing → compilation → assembly → linking) was documented, with a focus on the compilation stage where syntactic and semantic errors are detected.

Week 2 conducted the literature survey summarised in Section 2. The research gap was formally identified: no existing lightweight tool provides classification confidence, explanation, and step-by-step fix for GCC/G++ C++ errors in a single local pipeline.

Weeks 3–4: SRS and Architecture

Week 3 produced the Software Requirements Specification, defining 7 functional requirements, 8 non-functional targets, and the 8 error categories. Week 4 produced the architecture diagram, module interaction map, and finalised the Python + scikit-learn technology stack.

8 Error Categories Defined:

#	Category	Typical GCC Message Pattern
1	Missing Semicolon	expected ';' before ...
2	Undeclared Variable	'x' was not declared in this scope
3	Type Mismatch	invalid conversion from ... to ...
4	Unmatched Braces	expected '}' at end of input
5	Unmatched Parentheses	expected ')' before ...
6	Missing Return Statement	control reaches end of non-void function
7	Invalid Syntax in Declarations	unknown type name ...
8	Missing Header Include	'func' was not declared in this scope

Phase 3: Data Collection and Feature Extraction (Weeks 5–6)

Week 5: Error Collection

82 C++ source files were hand-crafted, each designed to trigger one of the 8 error categories. The files were compiled in batch using `collect_errors.py`, which invokes GCC/G++ via Python's `subprocess` module and captures both `stdout` and `stderr`. The resulting `collected_errors.csv` contains 129 rows (some files produced multiple errors), with columns for filename, raw error text, line number, error

keyword, and ground-truth category label.

Dataset Distribution:

Error Category	Samples	Percentage
Missing Semicolon	18	13.95%
Undeclared Variable	17	13.18%
Type Mismatch	16	12.40%
Unmatched Braces	16	12.40%
Unmatched Parentheses	15	11.63%
Missing Return Statement	17	13.18%
Invalid Syntax in Declarations	15	11.63%
Missing Header Include	15	11.63%
Total	129	100%

Week 6: Feature Extraction

`extract_features.py` processes each raw error string and produces a 28-dimensional feature vector. Features are grouped into four categories:

- **Keyword presence (binary):** Whether the error message contains keywords such as `expected`, `declared`, `conversion`, `control`, `unknown`, etc. (14 features).
- **Symbol presence (binary):** Whether specific symbols appear in the message: `;`, `{`, `}`, `(`, `)`, `<`, `>` (7 features).
- **Structural metrics (numeric):** Error line number, error column, message length, number of tokens in the message (4 features).
- **Error type tag (binary):** Whether the message is labelled `error` vs. `warning` by GCC (1 feature), plus 2 additional derived features.

The resulting `features_dataset.csv` has 129 rows and 32 columns (28 features + filename + raw error + label + line number).

Phase 4: Dataset Preparation and Model Training (Weeks 7–8)

Week 7: Dataset Preparation

The 129-row dataset was labelled with integer category codes 0–7 using `label_mapping.json`. The data was split 80/20 into training (103 samples) and test (26 samples) sets using stratified sampling to preserve class proportions. The split was fixed with a random seed for reproducibility.

Week 8: Model Training

Four classifiers were trained and evaluated on the test split:

Model	Test Accuracy	CV Mean	CV Std
Random Forest (100 trees)	86%	87.7%	$\pm 3.2\%$
Decision Tree	80%	81.4%	$\pm 4.8\%$
SVM (RBF kernel)	82%	83.1%	$\pm 3.9\%$
Naive Bayes (Gaussian)	74%	75.6%	$\pm 5.1\%$

The Random Forest model was selected as the best performer on both test accuracy and 5-fold cross-validation mean. It was serialised to `best_model.pkl` using Python's `pickle` module.

Why Random Forest? The ensemble approach mitigates overfitting on a small dataset (129 samples) by averaging across 100 decision trees. The model also provides out-of-bag probability estimates, which were used as the confidence score displayed to the user.

Phase 5: Classification Integration and Fix Engine (Weeks 9–10)

Week 9: Classification Integration

`classify_errors.py` provides the `classify(error_text)` function. It calls `extract_features()` to obtain the feature vector, loads `best_model.pkl` (cached after first load), and calls `model.predict_proba()` to obtain per-class probabilities. The predicted class with maximum probability is returned along with the confidence percentage.

Week 10: Fix Suggestion Engine

`fix_suggestions.py` defines a Python dictionary keyed on category name. Each entry contains:

- `explanation`: A 1–2 sentence plain-English description of the error cause.
- `fix_steps`: An ordered list of repair instructions.
- `code_example`: A before/after code snippet showing the wrong and correct versions.

`main.py` ties all modules together. It accepts a `.cpp` filename as a command-line argument, invokes GCC/G++, captures output, calls the classifier, retrieves the fix suggestion, and prints the complete diagnosis in a structured format.

Sample Output (TC-01):

```

=== AI-Assisted Compiler Error Diagnosis ===
File           : test_ms_new1.cpp
Raw Error      : test_ms_new1.cpp:6:16: error: expected ';' before 'double'

Predicted Category : Missing Semicolon
Confidence         : 97.2%
```

Explanation : A statement must end with a semicolon (;). The compiler expects one after the expression on line 6.

Fix Steps:

1. Go to line 6
2. Add ';' at the end of the statement
3. Recompile

Code Example:

```
Wrong : int x = 10
Correct: int x = 10;
```

=====

Phase 6: Security, Robustness, and Performance (Weeks 11–12)

Week 11: Security and Robustness Testing

18 security and robustness tests were executed. The test suite covered: empty input files, files with no errors, files with multiple simultaneous errors, files containing shell metacharacters in variable names, extremely long error messages, and files with non-ASCII comments. All 18 tests passed. No dangerous shell injection patterns were identified — the system uses Python's `subprocess` with argument lists (not shell strings), preventing command injection entirely.

Week 12: Performance Optimisation

Performance profiling identified model loading as the primary latency source. `best_model.pkl` is now cached in memory after the first load, reducing repeated-call latency from approximately 800 ms to under 50 ms. End-to-end response time (compile + extract + classify + suggest) is under 1 second for all test files. Peak memory usage was measured at under 150 MB, well within the 500 MB SRS target.

Final accuracy after optimisation: test set 86%, cross-validation mean 87.7%.

Week 13: System Testing and Validation

End-to-End Test Summary

16 unseen C++ programs (not seen during training) were tested. Results by group:

Error Category	Tests	Passed	Failed	Pass Rate
Missing Semicolon	2	2	0	100%
Undeclared Variable	2	2	0	100%
Type Mismatch	2	2	0	100%
Unmatched Braces	2	2	0	100%
Unmatched Parentheses	2	1	1	50%
Missing Return Statement	2	2	0	100%

Error Category	Tests	Passed	Failed	Pass Rate
Invalid Syntax in Declarations	2	2	0	100%
Missing Header Include	2	2	0	100%
TOTAL	16	15	1	93.75%

Single Failure Analysis

TC-10 (`test_up_new2.cpp`) was misclassified as Unmatched Braces instead of Unmatched Parentheses. The error message `expected ')' before ';' token` shares the keyword `expected` with Unmatched Braces messages, and the confidence score of 79.2% was the lowest in the entire test suite, indicating model uncertainty at this boundary. This is a known limitation and is documented as a future improvement item.

Overall Validation Against SRS Targets

All 8 SRS-defined targets were met or exceeded (see Section 3, Table 3.2). The system is confirmed as validated and ready for submission.

Source Code Overview

The repository is organised as follows:

```
AI-Assisted-Compiler-Error-Diagnosis-System/
|--- PLAN_and_IDEAS/           # Initial planning documents
|--- WEEK 1/                   # Problem definition report
|--- WEEK 2/                   # Literature survey
|--- WEEK3/                    # SRS document
|--- WEEK4/                    # Architecture diagram
|--- WEEK5/error_collection/    # collect_errors.py + 82 .cpp files
|                               # + collected_errors.csv (129 rows)
|--- WEEK6/                    # extract_features.py
|                               # + features_dataset.csv (129 x 32)
|--- WEEK7/                    # Dataset split + label_mapping.json
|--- WEEK8/                    # train_model.py + best_model.pkl
|--- WEEK9/                    # classify_errors.py
|--- WEEK10/                   # fix_suggestions.py + main.py
|--- WEEK11/                   # Security test suite (18 tests)
|--- WEEK12/                   # Performance profiling results
|--- WEEK13/                   # 16 end-to-end test programs
|                               # + validation report
```

Language breakdown: Python 69.6%, C++ 30.4%.

Primary source files: `main.py`, `classify_errors.py`, `fix_suggestions.py`, `extract_features.py`, `train_model.py`, `collect_errors.py`.

Conclusion and Future Work

Conclusion

The AI-Assisted Compiler Error Diagnosis System was successfully designed, implemented, and validated over 14 weeks. The system meets or exceeds all 8 SRS-defined non-functional requirements. A Random Forest classifier trained on 129 real GCC/G++ errors achieves 86% test accuracy and 87.7% cross-validation mean. End-to-end testing on 16 unseen programs yields a 93.75% pass rate. The system transforms raw, cryptic compiler messages into structured, beginner-friendly diagnoses including category, confidence, explanation, and step-by-step fix instructions.

Key Contributions

- A hand-crafted dataset of 129 real C++ compiler errors across 8 categories.
- A 28-feature extraction pipeline for GCC/G++ error messages.
- A trained Random Forest classifier with 86% accuracy, delivered as a serialised model.
- An end-to-end diagnosis pipeline operational in under 1 second.
- Complete documentation across 14 weekly phases with reproducible results.

Limitations

- Dataset size (129 samples) is small; larger datasets would improve generalisation.
- The boundary between Unmatched Braces and Unmatched Parentheses remains ambiguous in some cases (TC-10).
- The system handles single-error files well but multi-error files may produce less precise fixes.

Future Work

- Expand the dataset to 500+ samples per category using automated C++ error generation.
- Explore fine-tuned transformer models (CodeBERT) for improved accuracy on boundary cases.
- Add support for runtime errors (segmentation faults, memory leaks via Valgrind output).
- Build a web-based front end so students can submit code through a browser.
- Extend support to Python and Java compiler/interpreter errors.

Source Module Descriptions

File	Description
main.py	Entry point. Accepts .cpp file, runs pipeline, prints diagnosis.
collect_errors.py	Batch compiler; writes collected_errors.csv.
extract_features.py	Regex parser; 28-feature vector extraction.
train_model.py	Trains 4 models; saves best as best_model.pkl.
classify_errors.py	Loads model; returns (category, confidence%).
fix_suggestions.py	Lookup table: category → explanation + fix steps + example.
best_model.pkl	Serialised Random Forest (scikit-learn, 100 estimators).
label_mapping.json	Maps integer labels 0–7 to category name strings.
collected_errors.csv	129-row raw error dataset.
features_dataset.csv	129-row × 32-column feature dataset.

SRS Non-Functional Requirements — Full Table

All targets defined in Week 3 SRS, with Week 12 and Week 13 achieved values:

Metric	Target	Achieved	Status
Classification Accuracy (test set)	$\geq 85\%$	86%	PASS
Cross-Validation Mean Accuracy	$\geq 85\%$	87.7%	PASS
Response Time (per file)	< 3 seconds	< 1 second	PASS
End-to-End Test Pass Rate	$\geq 85\%$	93.75% (15/16)	PASS
Memory Usage	< 500 MB	< 150 MB	PASS
Robustness Tests	18/18 pass	18/18 (100%)	PASS
Security — Dangerous Patterns	0 patterns	0 patterns	PASS

Metric	Target	Achieved	Status
Error Categories Supported	8 categories	8 categories	PASS